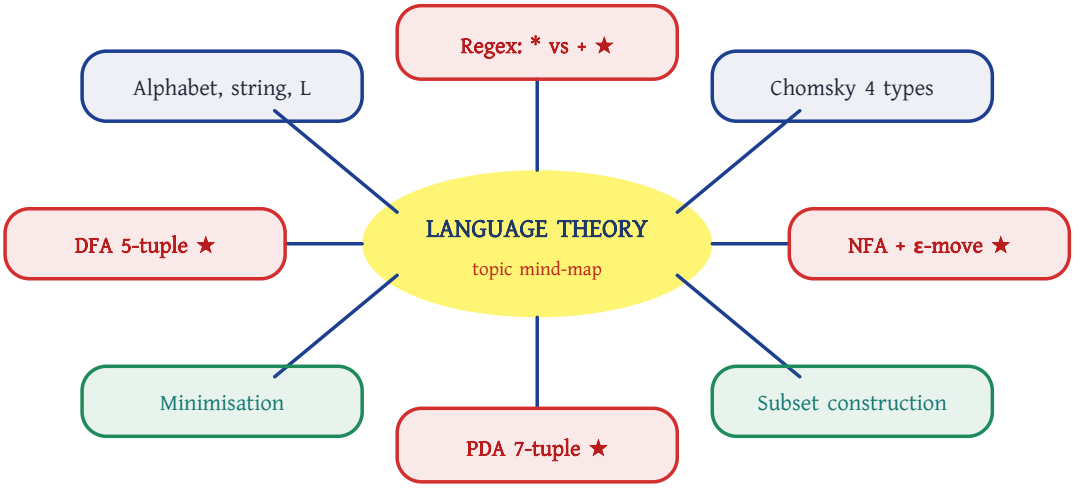
Language Theory & Finite Automata

Alphabet & language · Chomsky types · DFA · NFA · DFA = NFA · PDA: exam-ready in 11 sheets



↑ the whole topic on one page: red = high-yield, green = method / applied

Syllabus coverage

"Language theory; Finite automata: **deterministic finite automata**, **nondeterministic finite automata**, **equivalence and conversion** of deterministic and nondeterministic finite automata, **pushdown automata**; Context free languages; Context free grammars; Turing Machines: basic machines, configuration, computing with Turing machines." This notebook covers the language basics, the finite automata and the PDA; grammars are in the CFL & CFG lesson, Turing machines in the next one.

High-yield list

① DFA 5-tuple, PDA 7-tuple
② complete DFA: exactly 1 move
③ DFA ≡ NFA in power
④ subset build ≤ 2^n states
⑤ PDA = context-free
⑥ pumping lemma: NOT regular

Legend: **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

Contents

✓ Alphabet, string, language ..... p. 2
✓ Chomsky's four types ..... p. 3
✓ FA 5-tuple & the DFA ..... p. 4
✓ NFA and ε-moves ..... p. 5
✓ DFA = NFA, subset construction ..... p. 6
✓ Minimisation, pumping lemma ..... p. 7
✓ Pushdown automaton ..... p. 8
✓ DFA vs NFA vs PDA ..... p. 9
✓ Night-before sheet ..... p. 10
✓ Practice MCQs + written ..... p. 11

1 Alphabet, String, Language

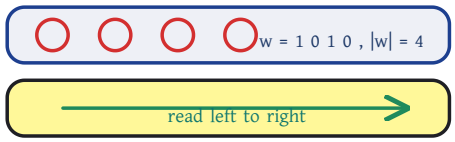
Definition: **Language theory** studies **formal languages**: sets of strings built from a fixed finite alphabet. A machine **accepts** a language if it says "yes" for **exactly** its strings and "no" for all others. Everything else in this chapter is a machine for deciding one such set.

Term	Meaning	Example with $\Sigma = \{0, 1\}$
Alphabet Σ	finite, non-empty set of **symbols**	$\Sigma = \{0, 1\}$
String w	finite sequence of symbols; $\lvert w\rvert$ = length	w = 1010, $\lvert w\rvert = 4$
ε	the **empty string**, $\lvert\varepsilon\rvert = 0$, $\varepsilon \in \Sigma^*$	no symbols at all
Σ^*	all strings over Σ, ε included	{ε, 0, 1, 00, 01, 10, 11, ...}
Language L	any subset of Σ^*	L = {w | w ends with 0}
Σ^n	strings of length exactly n; $\lvert\Sigma^n\rvert = \lvert\Sigma\rvert^n$	Σ^2 = {00, 01, 10, 11}

Regular expressions: four operators

Operator	Means	Reads as
a + b	union: a **or** b	"plus = or"
ab	concatenation: a then b	"written together"
(a + b)*	Kleene star: **zero** or more	"any number"
(a + b)+	one or more	"at least one"

```
(a+b)*  = a*b* = (a+b)+ U {e}
(a+b)+  = Sigma* \ {e}
ab*a    = {a, aa, aba, abba, ...}
```



arrow = head, box = memory

a string is just symbols under one head

Memory trick: **star = zero or more, plus = one or more.** So $(a+b)^*$ **includes ε** and $(a+b)+$ does not.

Exam trap: ab^*a = an a, then **zero** or more b's, then an a — so with zero b's the string is **aa**. And "w **contains** 01" ≠ "w **ends with** 01": a finite automaton can remember a suffix, never an arbitrary earlier occurrence.

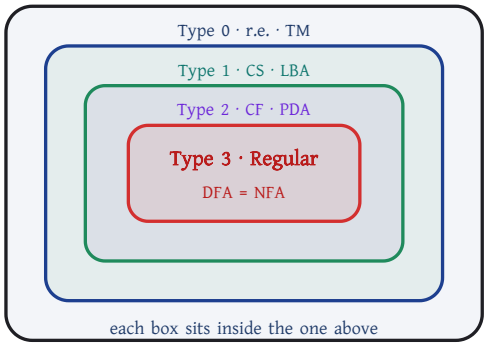
Operations on languages: Languages are sets, so they take **set** operations: $L_1 \cup L_2$, $L_1 \cap L_2$, $L^c = \Sigma^* - L$ (always over the same Σ), $L_1 \cdot L_2 = \{xy \mid x \in L_1, y \in L_2\}$ and L^* (ε included). Complement is taken over Σ^*, never "over the machine": for $\Sigma = \{0,1\}$, the complement of "ends with 0" is "ends with 1 **or** is empty".

L is any subset of Σ* — including ∅ and Σ*

2 Chomsky's Hierarchy: Four Types

Definition: Chomsky's classification of formal languages into **four nested classes**, each defined by a kind of grammar and matched by a kind of machine. Type **n** sits inside Type **n-1**: every regular language is also context-free, and so on up the ladder.

Type	Grammar	Automaton	Example language
Type 0 recursively enumerable	unrestricted	Turing machine	{w # w | w ∈ {0,1}*}
Type 1 context-sensitive	linear bounded (LBA)	LBA	{a^n b^n c^n | n ≥ 1}
Type 2 context-free	context-free (CFG)	**pushdown automaton**	{a^n b^n | n ≥ 0}
Type 3 regular	regular	**finite automaton**	{w | w ends with 01}



strict containment: 0 ⊃ 1 ⊃ 2 ⊃ 3

Machine ↔ language, the one-line answer:
FA (DFA / NFA) ⇒ **regular** = Type 3 · PDA ⇒ **context-free** = Type 2
LBA ⇒ **context-sensitive** = Type 1 · TM ⇒ **recursively enumerable** = Type 0

Memory trick: Read the table **upwards**: each step adds one kind of memory — **none** (FA) → **stack** (PDA) → **tape** (LBA, TM). More memory ⇒ more languages.

Exam trap: {a^n b^n c^n} needs a **linear bounded** automaton, not a PDA: one stack counts only **one** balance, so it accepts a^nb^n but never $a^nb^nc^n$. Type 0's example needs the "#" to find the middle.

Neighbouring lessons: Context-free **grammars** (productions, derivation trees, parsing) are in the CFL & CFG lesson; Turing machines, configurations and computability in the Turing Machines lesson. Both name the same Type 2 and Type 1 languages that this chapter's automata accept.

IT link: The regex engines inside *grep*, *RegExp* and *lex* are finite automata in disguise: the **syntax** of a language is a regular (Type 3) language, while its **grammar** is context-free (Type 2) — which is exactly why parsing needs a stack.

3 Finite Automaton: the 5-tuple

Definition: A **finite automaton** $M = (Q, \Sigma, \delta, q_0, F)$ reads a string left to right, changing state at each symbol, then **accepts** or **rejects**. It has **no memory** beyond its current state — which is why it recognises only regular languages.

Part	Is	Meaning
Q	finite non-empty set of states, $\|Q\| = n$	the boxes in the diagram; $q_0 \in Q$
Σ	input alphabet (finite, non-empty)	the symbols the tape may carry
δ	transition function, $\delta: Q \times \Sigma \to Q$ (DFA)	the arrows: read a symbol, move state
q_0	the single **start** state	where the head begins
F	set of **final / accepting** states, $F \subseteq Q$	accept iff the run ends in F

Memory trick: Five parts in order: **Q**uestions (states), **Σ**ymbols, the **δ**ecision rule, the quest**0** entry, the **F**inish states. A PDA adds **Γ** (stack alphabet) and **Z_0** (bottom marker) = **7**.

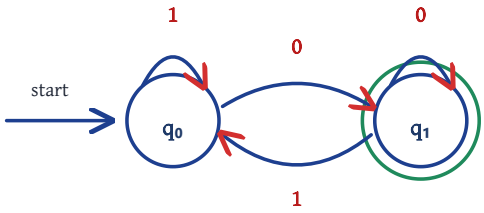
4 Deterministic Finite Automaton (DFA)

Definition: For **every** pair (state, symbol) there is **exactly one** next state: $\delta: Q \times \Sigma \to Q$ is **total**. No ε-move, no guessing.

L = {w | w ends with 0}

State	on 0	on 1	meaning
→ q_0	q_1	q_0	last symbol was 1
* q_1	q_1	q_0	last symbol was 0

over $\Sigma = \{0,1\}$. → start · * accepting. **1100:** q_0 -1→ q_0 -1→ q_0 -0→ q_1 -0→ $q_1 \in F \Rightarrow$ **accept**. **1111:** ends in $q_0 \notin F \Rightarrow$ reject.



one move on 0 and on 1, from every state

"w ends with 0" — q_1 double circle = accepting

Accept / reject, formally: Extend δ to strings: $\delta^*(q_0, w)$ = the state reached after the **whole** w is read, so **$w \in L \Leftrightarrow \delta^*(q_0, w) \in F$**. A missing move means rejection — which is why a blank cell needs a trap state.

Exam trap: A DFA has **exactly one** transition per (state, symbol); "zero or more" and ε-moves belong to an **NFA**. A blank cell means an **incomplete** machine — add the trap state d.

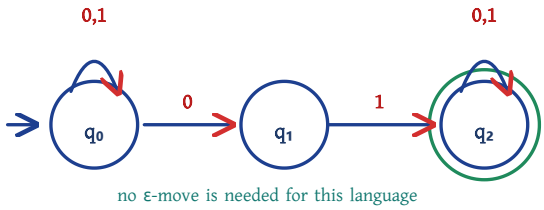
Memory trick: Input **1** separates q_0 from q_1, so this 2-state DFA is already **minimal**.

5 Nondeterministic Finite Automaton (NFA)

Definition: For a (state, symbol) pair an NFA may have **zero, one or many** next states — $\delta: Q \times (\Sigma \cup \{\varepsilon\}) \to P(Q)$ — and it may take an **ε-move**: change state **without consuming input**. It accepts if **some** path reaches a final state when the input runs out.

L = {w | w contains 01}

From	on 0	on 1
→ q_0	q_0 **and** q_1	q_0
q_1	—	q_2
* q_2	q_2	q_2



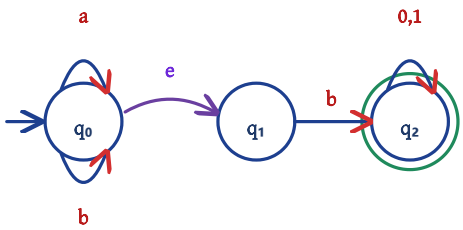
NFA for "contains 01": q_2 loops, so the memory survives

q_0 guesses "a 0 may have just arrived"; a following 1 confirms it and q_2 keeps that memory for good. **0110:** q_0 -0→ $\{q_0,q_1\}$ -1→ $\{q_0,q_2\}$ -1→ $\{q_0,q_2\}$ -0→ $\{q_0,q_2\}$ ∋ q_2 ⇒ accept. **110:** q_0 -1→ q_0 -1→ q_0 -0→ $\{q_0,q_1\}$ ⇒ reject.

When an ε-move really helps: L = {w | w contains ab}

The machine must not commit to the position of "ab", so it runs **two branches at once**: q_0 keeps scanning with a 0,1 loop, and the ε-move lets a copy jump to q_1, whose only job is to expect the b that confirms the a. An ε-move is the NFA's way of **cloning** — it costs no input, only a state change.

```
q0 --a--> q0     q0 --b--> q0
q0 --e--> q1     q1 --b--> q2*
q2 --0,1--> q2
```



the ε-move (purple) copies the head forward for free

Exam trap: "zero or more transitions" and "ε allowed" are **NFA** features. An NFA with no ε-move and one move per pair is a **DFA** written differently — which is the whole content of the next sheet.

Worked trace ✍️ the ε-NFA on "aab": start $\{q_0\}$ → a: q_0 -a-> q_0 plus the ε-copy q_1 ⇒ $\{q_0, q_1\}$ · a: only q_0 has an a-move ⇒ $\{q_0, q_1\}$ · b: q_0 -b-> q_0 and q_1 -b-> q_2 ⇒ $\{q_0, q_2\}$ ∋ q_2 ⇒ **accept**. A branch with no defined move simply dies — the "zero transitions" case.

How an NFA is really run: Carry a **set** of states, never a single one: start with $\{q_0\}$; on each symbol a, replace the set by the union of the successors of **every** member; accept iff the final set meets F. That is why the trace above reads $\{q_0,q_1\}$ then $\{q_0,q_2\}$ — and it is exactly the object the next sheet turns into a DFA state.

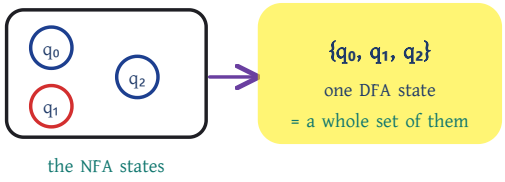
6 Equivalence of DFA and NFA

Theorem (high-yield): **DFA and NFA have exactly the same expressive power**: both accept **exactly** the regular languages. So **DFA → NFA is trivial** (an NFA may have one move per pair, so a DFA already *is* an NFA), **NFA → DFA is always possible**, an NFA can be **exponentially smaller**, and a DFA runs **faster** because there is no guessing to backtrack.

Proof sketch (both directions): **DFA ⇒ NFA:** keep the same states and arrows — the NFA definition already allows zero or one move per pair. **NFA ⇒ DFA:** let one DFA state stand for the whole **set** of NFA states the NFA could occupy after the same prefix; then w is accepted iff that set holds an accepting state. ■

Subset (powerset) construction

1. One DFA state = a **set** of NFA states; start with ε-closure(q_0) = q_0 plus every state reachable by ε-moves alone
2. **next = ε-closure(move(S, a))** — the union of the successors of **every** member of S; a set accepts if it **contains** an accepting NFA state, and the empty set ∅ is the dead state



each DFA state is a box of NFA states

Size bound: An n-state NFA gives at most 2^n DFA states (2^n subsets), and the bound is **tight** — some languages force every subset to appear.

Exam trap: $\{q_0, q_2\}$ and $\{q_0, q_1, q_2\}$ in the table below have **identical** moves, so they merge: the minimal DFA has **3** states, not 4.

Worked conversion ✍️: the "contains 01" NFA:

DFA state = a set	on 0	on 1	accepting?
→ $\{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$	no
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$	no
$\{q_0, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_0, q_2\}$	yes
$\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_0, q_2\}$	yes

Trace 0101: $\{q_0\}$ -0→ $\{q_0,q_1\}$ -1→ $\{q_0,q_2\}$ -0→ $\{q_0,q_1,q_2\}$ -1→ $\{q_0,q_2\}$ ⇒ accept (four sets, far below $2^n = 16$).

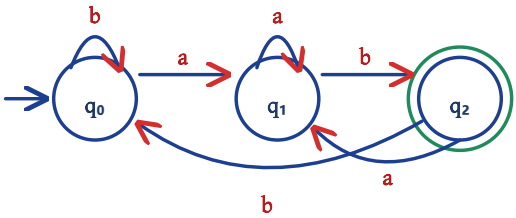
IT link: *grep -E* builds an **NFA**, simulates all its states at once (Thompson), then converts it to a DFA table: cheap description, fast machine.

Memory trick: NFA = "maybe, at once"; DFA = "exactly this one". Count the tuple to name the machine: **5** = FA, **7** = PDA.

7 Minimisation & the Pumping Lemma

Minimising a DFA (table filling)

1. Drop **unreachable** states (no path from q_0)
2. Mark every (accepting, non-accepting) pair, then repeat: mark (p, q) if some symbol sends them to a marked pair
3. Unmarked pairs are **indistinguishable** ⇒ merge, then rebuild the table; the answer is **unique up to renaming**



minimal DFA for "w ends in ab": q_0 nothing yet, q_1 last was a, q_2 last two were ab (accepting)

Pumping lemma (regular languages)

If L is regular then some length p exists such that **every** $w \in L$ with $|w| \geq p$ splits as $w = xyz$ with $|xy| \leq p$, $|y| > 0$ and $xy^nz \in L$ **for every** $n \geq 0$. Its only use is the contrapositive: find one w that fails to pump ⇒ **L is not regular**.

Exam trap: Satisfying the lemma does **not** prove regularity — some non-regular languages also satisfy it. "Prove it is regular" is not a pumping-lemma question.

Language over {a,b}	Regular?	Machine / states
a* b* · (ab)*	yes	DFA, 2–3 states
even number of a's	yes	DFA, 2 states (parity)
w ends in ab	yes	DFA, 3 states
$\{a^n b^n \mid n \geq 0\}$	**no**	PDA needed
$\{w \mid \#a = \#b\}$	**no**	PDA needed
$\{a^n b^n c^n \mid n \geq 1\}$	**no**	not even CF — LBA / TM

Worked example ✍️ binary divisible by 5: Read a binary number MSB first; accept when its value ≡ 0 (mod 5), with ε counting as the value 0. Use **one state per remainder** $r \in \{0,1,2,3,4\}$ and move $r \to (2r + b) \bmod 5$: $r = 0$: $0\to0$, $1\to1$ · $r = 1$: $0\to2$, $1\to3$ · $r = 2$: $0\to4$, $1\to0$ · $r = 3$: $0\to1$, $1\to2$ · $r = 4$: $0\to3$, $1\to4$. State 0 is both the **start** and the only **accepting** state.

Trace 1111 (= 15): 0 -1→ 1 -1→ 3 -1→ 2 -1→ 0 ⇒ accept, since 15 mod 5 = 0.

5 states, and minimal: 2 is invertible mod 5, so powers of 2 reach every non-zero residue and any two different remainders can be separated by some suffix of 1s.

8 Pushdown Automaton (PDA)

Definition: A **pushdown automaton** is a finite automaton plus a **stack** (LIFO memory) — $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$. Each move reads at most one input symbol and one stack symbol, then pushes or pops. One stack remembers **one unbounded count**, so a PDA accepts **exactly the context-free** languages.

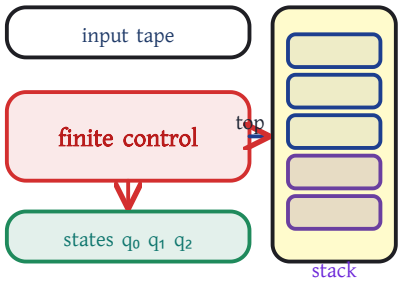
Part	Meaning
Q, Σ, q_0, F	exactly as in the FA 5-tuple: states, input alphabet, start state, final states
Γ (new)	the **stack alphabet**: what may sit in the stack
Z_0 (new)	bottom marker, $Z_0 \in \Gamma$; the stack begins as Z_0 and is "bare" when only Z_0 is left
δ	δ: $Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \to P(Q \times \Gamma^*)$ — read a symbol (or ε) and a stack top, then replace the top by a **string**

$L = \{a^n b^n \mid n \geq 0\}$, with $\Sigma = \{a, b\}$ and $\Gamma = \{A, Z_0\}$

state	input	top	replaces top	role
q_0	a	Z_0	A Z_0	push for the first a
q_0	a	A	A A	push for every further a
$q_0 \to q_1$	b	A	ε	first pop — now counting down
q_1	b	A	ε	keep popping
q_1	ε	A	ε	input over and stack bare ⇒ accept
$q_0 \to q_2$	ε	Z_0	Z_0	n = 0: also accept the empty input

```
input: a a a b b b
stack: Z0 -> AZ0 -> AAZ0 -> AAAZ0 -> AAZ0 -> AZ0 -> Z0  (q2, accept)
```

Exam trap: The five moves above, on their own, **reject** the empty string — from q_0 with a bare stack there is no ε-move, yet the language reads $\{a^n b^n \mid \mathbf{n \geq 0}\}$. So add $\delta(q_0, \varepsilon, Z_0) \to (q_2, Z_0)$, as in the last row, or accept by empty stack. **n ≥ 0 in the statement and no ε-move out of q_0 cannot both be true.**



finite control + one stack

Memory trick: Push on the a's, pop on the b's, accept only when the stack is **bare**. "Pushes must equal pops."

Common misconception: A **deterministic** PDA does **not** match a nondeterministic one — the opposite of DFA vs NFA: $\{a^n b^n\}$, $\{w\ c\ w^R\}$ are deterministic, $\{w\ w^R\}$ needs an **NPDA**.

9 DFA vs NFA vs PDA

Feature	DFA	NFA	PDA
Formal definition	5-tuple $(Q, \Sigma, \delta, q_0, F)$	5-tuple $(Q, \Sigma, \delta, q_0, F)$	**7-tuple** $(Q, \Sigma, \mathbf{\Gamma}, \delta, q_0, \mathbf{Z_0}, F)$
Transitions per input	**exactly 1**	0, 1 or more	0, 1 or more
ε-transitions	not allowed	allowed	allowed
Memory	none (state only)	none (state only)	**one stack**
Language class	regular (Type 3)	regular (Type 3)	context-free (Type 2)
Power compared	DFA and NFA are **equal** — mutually convertible		strictly stronger than any FA
Deterministic form	by definition	nondeterministic	**DPDA < NPDA** — not equal
Machine size / speed	can be exponentially larger, but fastest	often tiny; needs backtracking	states + stack; slower
Example language	w ends with 0	w contains 01	$a^n\ b^n$, $w\ w^R$

Which machine for which language

Language	Smallest model
$a^* b^* \cdot$ even number of a's	DFA, 2 states
w ends in ab	**DFA, 3 states**
$\{a^n b^n \mid n \geq 0\}$	**DPDA** — deterministic CF
$\{w\ c\ w^R \mid w \in \{a,b\}^*\}$	**DPDA** — deterministic CF
$\{w\ w^R \mid w \in \{a,b\}^*\}$	**NPDA only** — no DPDA
$\{a^n b^n c^n\}$	neither — LBA / TM

The ladder of power

Finite automaton ⊂ pushdown automaton ⊂ linear bounded automaton ⊂ Turing machine. Every step up adds memory and accepts **strictly more**: no PDA accepts a non-context-free language, no FA accepts $a^n b^n$.

Memory trick: "**FA** = **f**ixation (one state), **PDA** = **p**ile (a stack), **TM** = **t**ape (unbounded, read and write)."

IT link: A text editor matches brackets with a **stack** — the PDA of sheet 8. Whether a *while/if* nests correctly is **not** a finite-automaton property.

Exam trap: The inequality students forget: **DPDA ≠ NPDA**, while **DFA = NFA**. Both statements appear in the same paper.

Night-Before-the-Exam Sheet

Fact sheet:

Alphabet Σ, string w, |w|, ε, Σ*, L ⊆ Σ*

$|\Sigma^n| = |\Sigma|^n \cdot \Sigma^0 = \{\varepsilon\}$

star * = zero or more; plus + = one or more

(a+b)* = Σ* (with ε); (a+b)+ excludes ε

ab*a = a, zero or more b, then a → "aa" too

Type 0 ⊃ 1 ⊃ 2 ⊃ 3 (TM, LBA, PDA, FA)

FA / DFA / NFA = 5-tuple $(Q, \Sigma, \delta, q_0, F)$

PDA = 7-tuple + Γ (stack alphabet) + Z_0 (bottom)

complete DFA: exactly 1 move per (state, symbol)

NFA: 0 or more moves; ε-move allowed

DFA ≡ NFA; subset build ≤ 2^n states, tight

DPDA < NPDA — the exception, not the rule

minimal DFA "ends in ab" = 3 states

binary mod 5 (MSB first) = 5 states, r → (2r+b) mod 5

pumping lemma proves **NOT** regular, never regular

ε ∈ $\{a^n b^n \mid n \geq 0\}$: the PDA accepts the empty string

Top 12 one-liners for MCQs

1. DFA = 5-tuple, PDA = 7-tuple

7. $\{a^n b^n\}$ is **not** regular — PDA needed

2. Complete DFA ⇒ exactly one transition
3. DFA and NFA accept the same languages
4. Subset construction ≤ 2^n DFA states
5. PDA ⇒ context-free, nothing more
6. star allows zero, plus does not
8. "ends in ab" needs 3 states
9. binary divisible by 5 needs 5 states
10. Pumping lemma only disproves regularity
11. $\{w\ w^R\}$ needs nondeterminism
12. $\varepsilon \in \{a^n b^n \mid n \geq 0\}$

Which machine does the question want?: Mentions **states, no memory, tuple of 5** ⇒ DFA · "guess", **several moves** or **ε** ⇒ NFA · **stack**, tuple of **7**, language $a^n b^n$-style ⇒ PDA · needs **two stacks / a tape** or not context-free at all ⇒ TM.

Top traps: "zero or more transitions" is an NFA feature · a blank cell in a DFA table means an incomplete machine — add the trap state · satisfying the pumping lemma does not prove regularity · a subset-built DFA need not be minimal · DFA = NFA but DPDA ≠ NPDA · $a^n b^n$ with $n \geq 0$ includes the empty string

All tricks together: "star = zero or more, plus = one or more" · "five letters, then Γ and Z_0" · "DFA must know, NFA may guess" · "pushes must equal pops" · "FA = fixation, PDA = pile, TM = tape" · "one stack counts one balance"



each step up = one more kind of memory

equal at the first step, strict at every step after it

✓ Practice MCQs (exam pattern, 10 marks)

1. A DFA is a tuple of how many parts?
   (a) 3 (b) 4
   (c) 5 (d) 7
2. In a **complete** DFA, each state has:
   (a) exactly 1 move (b) at most 1
   (c) 0 or more (d) at least 2
3. An NPDA accepts exactly the:
   (a) regular langs (b) context-sensitive
   (c) r. enumerable (d) context-free
4. Over {a, b}, (a + b)* denotes:
   (a) all, with ε (b) non-empty
   (c) #a = #b (d) $a^n b^n$ only
5. Which one is **NOT** regular?
   (a) a* b* (b) (ab)*
   (c) even # of a (d) $\{a^n\ b^n \mid n \geq 0\}$
6. Which string is in ab*a?
   (a) ab (b) aa
   (c) ba (d) abab
7. An n-state NFA gives at most:
   (a) n states (b) 2n states
   (c) n^2 states (d) 2^n states
8. Minimal DFA for "ends in ab":
   (a) 2 states (b) 3 states
   (c) 4 states (d) 5 states

9. The pumping lemma proves a language is:

(a) minimal (b) convertible

(c) NOT regular (d) regular

10. Minimal DFA for binary divisible by 5:

(a) 5 states (b) 2 states

(c) 4 states (d) 32 states

Answer key → 1-c · 2-a · 3-d · 4-a · 5-d · 6-b · 7-d · 8-b · 9-c · 10-a

Written question: Design a DFA over {0, 1} that accepts exactly the strings ending in **01**. Give its transition table and trace it on *1101*. 8 marks

Model answer ✍️:

1. **States** (progress toward 01): q_0 start, nothing matched; q_1 last symbol was 0; q_2 last two were 01 — **accepting** 2
2. **Table**, one move per symbol:

state	on 0	on 1
→ q_0	q_1	q_0
q_1	q_1	q_2
* q_2	q_1	q_0

   $M = (\{q_0, q_1, q_2\}, \{0,1\}, \delta, q_0, \{q_2\})$ 3
3. **Trace 1101:** q_0 -1→ q_0 -1→ q_0 -0→ q_1 -1→ $q_2 \in F \Rightarrow$ **accepted**; minimal, since q_0 and q_1 differ on input 1 3

Template to memorise: a **k**-symbol suffix always needs **k+1** states: q_0 nothing matched … q_k the whole suffix just read (accepting).

Before the exam ✓

- ✓ Can write both tuples from memory
- ✓ Can draw DFAs and NFAs for "ends with" / "contains"
- ✓ Can run subset construction on a 3-state NFA

Also practise (written):

① minimise a 5-state DFA

② convert the "contains ab" ε-NFA to a DFA